

Current OS, Processes, Linux and Utilities — Detailed Lecture

Notes

JKSSB Computer Awareness

October 8, 2026

60.1 Deadlock and its four necessary conditions

A **deadlock** is a situation in which a group of processes are each waiting for a resource that is held by another process in the group, so that none of them can ever proceed. A simple picture is two processes: process A holds a printer and waits for a scanner, while process B holds the scanner and waits for the printer. Each waits for the other, and both stay stuck forever.

For a deadlock to be possible, four conditions must hold *at the same time*. They are usually listed as follows.

Condition	Meaning
Mutual exclusion	A resource is held by only one process at a time.
Hold and wait	A process holds at least one resource while waiting for more.
No preemption	A resource cannot be forcibly taken away from its holder.
Circular wait	A cycle of processes exists, each waiting for a resource held by the next.

Definition (Deadlock). *A state in which a set of processes are each blocked waiting for a resource held by another process in the same set, so no process can make progress.*

Definition (No preemption (as a condition)). *The rule that a resource already allocated to a process cannot be taken away by force; it must be released voluntarily. Its opposite, **preemption**, is used to recover from a deadlock.*

60.2 Deadlock recovery by resource preemption

Once a deadlock has been detected, the operating system must break it. One recovery method is **resource preemption**: the system takes a resource away from one of the processes and gives it to another process that needs it, breaking the circular wait. The chosen process is usually rolled back to a safe earlier state, or restarted. Preemption is also used to break the *no-preemption* condition directly, since it removes the guarantee that a resource stays with its holder.

Example 60.2.1 (The exam contrast). *An item asks for a method of recovering from deadlock. The correct answer is **resource preemption** — the system forcibly reclaims a resource. A distractor may name “mutual exclusion” or “circular wait”, but those are **necessary conditions** for a deadlock, not recovery methods. The other trap is to think preemption is one of the four conditions; the condition is actually **no preemption**. (PYQ-10)*

60.3 Processes and the scheduler

A **process** is a program in execution. When a program is launched, the operating system creates a process for it and keeps the information needed to run it — its state, its program counter, and the resources it holds. Because many processes are usually ready at once but only one can run on a given CPU core at a time, the operating system includes a **scheduler** that decides which ready process gets the CPU next.

Scheduling may be **non-preemptive**, where a running process keeps the CPU until it finishes or blocks, or **preemptive**, where the operating system can interrupt a running process and switch to another. Priorities are one of the most common ways to choose between processes, and it is the combination of *priorities* and *preemption* that produces the problems studied in the next sections: priority inversion and starvation.

Definition (Process). *A program in execution, together with the state and resources the operating system maintains for it.*

Definition (Scheduler). *The operating system component that selects which ready process is given the CPU next.*

60.4 Priority inversion versus priority inheritance

In a priority-based system, a task with a higher priority is supposed to run before one with a lower priority. **Priority inversion** breaks that rule. It happens when a **high-priority** task is blocked because a **low-priority** task currently holds a resource the high-priority task needs. While the low-priority task holds that resource, a **medium-priority** task can preempt it and run. The result is that the high-priority task ends up waiting behind the medium-priority task, even though it should have run first — its priority has effectively been inverted.

The standard fix is **priority inheritance**. While the low-priority task holds the resource, it temporarily **inherits** the high priority of the task that is waiting for it, so it runs at that high priority, releases the resource quickly, and then returns to its original low priority. This lets the high-priority task obtain the resource without waiting behind the medium-priority task.

Point	Priority inversion	Priority inheritance
What it is	A problem in which priority order is reversed.	A solution that temporarily raises the holder's priority.
Cause	A low-priority task holds a needed resource.	Applied while a resource is held and awaited.
Effect	High-priority task waits behind a medium-priority task.	Low-priority holder finishes fast, so the high-priority task proceeds.

Definition (Priority inversion). *A condition in which a high-priority task is indirectly preempted by a lower-priority task, typically because a low-priority task holds a shared resource.*

Example 60.4.1 (What does not fix inversion). *An item asks which method prevents priority inversion. **Round-robin** scheduling gives every ready process a turn but knows nothing about resource holding, so by itself it does **not** prevent inversion. The correct method is **priority inheritance** (or the related priority-ceiling protocol). (PYQ-11)*

60.5 Starvation and aging

Starvation (indefinite blocking) is different from deadlock. A starved process is *ready* to run and could run in principle, but it is **repeatedly passed over** by the scheduler, so it may wait for an unbounded time. A common case is a low-priority process in a strict priority system: as long as higher-priority processes keep arriving, the low-priority process never gets the CPU.

The standard remedy for starvation is **aging**: the system gradually **raises the priority** of a process the longer it waits, so that eventually it becomes the highest-priority ready process and runs.

Definition (Starvation). *A scheduling condition in which a ready process is indefinitely denied the resources it needs because other processes are repeatedly preferred.*

Point	Starvation	Deadlock
Can the process ever run?	Yes, in principle	No, not without outside action
Cause	Scheduling choices (long wait)	A circular resource dependency
Cure	Aging (raise priority with waiting time)	Detection + recovery (for example preemption)

60.6 Preemptive priority scheduling

In **priority scheduling**, the scheduler always picks the ready process with the **highest priority**. The scheme is **preemptive** when a process that is currently running can be **interrupted** as soon as a process with a higher priority becomes ready; the higher-priority process then runs. It is **non-preemptive** when the running process keeps the CPU until it either finishes or blocks for input/output. Preemptive priority scheduling gives quick service to important tasks, but with strict priorities it can starve low-priority tasks unless aging is added.

60.7 Real-time operating systems (RTOS)

An **RTOS** is a **Real-Time Operating System**. It is designed so that the system responds within a **bounded time**, allowing **deadlines** to be met. Correctness in a real-time system depends not only on the *result* but also on *when* the result is produced.

Definition (Real-time operating system (RTOS)). *An operating system designed to process data and events within guaranteed time limits, so that deadline requirements are satisfied.*

Real-time systems are classified by what happens when a deadline is missed. In a **hard real-time** system a missed deadline is a **failure** and can have serious or catastrophic consequences. In a **soft real-time** system a missed deadline only **degrades** the quality of the result; the result is still useful. The trap is to think an RTOS is simply “a very fast operating system”. Speed helps, but the defining property is **meeting deadlines**.

60.8 The Linux kernel

The **kernel** is the **core of the operating system**. It is the part that runs with full control of the hardware and provides the basic services every other program relies on. In Linux, the kernel

manages **processes** (through the scheduler), **memory**, **devices** (through device drivers), and **file systems**.

The kernel runs in a privileged mode called **kernel space**, while ordinary application programs run in **user space**. A user program asks the kernel for a service — reading a file, for example — through a **system call**; the kernel performs the operation and returns the result. The Linux kernel is **open-source**: its source code can be viewed, modified, and shared.

Definition (Kernel). *The central component of an operating system that manages the hardware resources (processes, memory, devices, and file systems) and mediates between applications and the hardware.*

The **shell** is frequently confused with the kernel. The shell is the *command interpreter* that a user types commands into; it is not the kernel. The shell passes the user’s requests to the kernel, which does the actual work.

60.9 /root versus /home

Linux stores the personal files of each user in that user’s **home directory**. The **superuser**, whose account name is **root**, has its home directory at **/root**. Ordinary users’ home directories are normally placed under **/home**, for example **/home/alice**.

It is important not to confuse two different things with similar names:

Path	Meaning
/	The root of the whole file system — the single top directory from which all other directories branch.
/root	The home directory of the root (superuser) account.
/home	The parent directory holding the home directories of ordinary users .

The trap is to read “/root” as “the root of the file system”. The top of the file system is / (a single slash). /root is only the superuser’s own home folder.

60.10 File systems

A **file system** is the method an operating system uses to organise, store, and retrieve files and folders on a storage device. Different systems use different file systems.

File system	Full form	Typical use
NTFS	New Technology File System	Default for modern Windows installations.
FAT32	File Allocation Table, 32-bit	Older system; widely compatible with small drives and memory cards.
ext4	Fourth Extended file system	Native and common default file system for Linux.

The exam cue is the pairing: **NTFS = Windows**, **ext4 = Linux**, and **FAT32 = broad compatibility**. Full forms are commonly asked directly.

60.11 Windows utilities: Store, WordPad/Notepad, Disk Cleanup

The **Windows Store** (now the **Microsoft Store**) is the built-in **online marketplace** in Windows. Users **search, download, install, and update** apps from it, so applications come from a trusted catalogue rather than from arbitrary websites.

Windows includes two simple text editors that are easy to confuse.

Point	Notepad	WordPad
Text handling	Plain text only; no formatting	Basic formatting: bold, italic, fonts, colour
Typical save	.txt	.rtf (rich text format)
Capability	The smallest, fastest text editor	Can insert pictures and objects

Notepad is a plain-text editor: it displays and saves characters without any formatting. **WordPad** supports *basic* rich-text formatting and can include pictures, but it is still lighter than a full word processor such as MS Word.

Disk Cleanup is a built-in Windows utility that **frees disk space**. It deletes files that are not needed, such as temporary files, thumbnail caches, downloaded program files, and the contents of the **Recycle Bin**. It is the tool to reach for when a drive is running short of free space.

60.12 Defragmenter and the SSD caution

On a traditional **hard disk drive (HDD)**, a file can become **fragmented**: its pieces are scattered in different places on the rotating disk, which makes reading slower. The **Disk Defragmenter** rearranges the pieces so that each file is stored **contiguously** (in one run), which improves access speed.

SSD caution. A **solid-state drive (SSD)** has **no moving parts**. Defragmenting it gives no speed benefit and only adds **unnecessary writes**, which reduce the drive's working life. SSDs should **not** be defragmented; Windows uses the **TRIM** command for SSDs instead.

Definition (Defragmentation). *The process of rearranging the fragments of files on a hard disk so that each file occupies contiguous space, improving read performance.*

60.13 Imaging Fax and the xcopy /s command

Windows Fax and Scan is the built-in **imaging and fax** utility in Windows. With a scanner it can **scan** a document and save it as an image; with a fax modem or fax service it can **send and receive faxes**. This is the tool the exam means by “Imaging Fax”.

xcopy is a Windows **command-line** command that copies files and whole directory trees. Its switch **/s** copies the named directory *including its subdirectories*, but it does not copy empty ones. Adding **/e** copies empty subdirectories as well. The exact form to remember is **xcopy /s**: “copy with subdirectories”.

Command / tool	What it does
xcopy /s	Copies files and subdirectories (empty ones excluded).
xcopy /s /e	Also copies empty subdirectories.
Windows Fax and Scan	Scans documents and sends/receives faxes (Imaging Fax).

60.14 Exam retrieval and common confusions

Six distinctions prevent most errors on this topic.

1. Deadlock's four conditions are mutual exclusion, hold and wait, **no preemption**, and circular wait. **Preemption** is a **recovery** method, not a condition (PYQ-10).
2. **Priority inversion** is the problem; **priority inheritance** is the cure. Round-robin by itself does **not** prevent inversion (PYQ-11).
3. **Starvation** is indefinite *scheduling* delay, not deadlock; **aging** fixes starvation.
4. An **RTOS** is defined by **meeting deadlines** (hard vs soft), not merely by being fast.
5. The Linux **kernel** is the core; the **shell** is the command interpreter. **/root** is the superuser's home; **/** is the top of the file system; **/home** holds ordinary users' homes.
6. **Notepad** = plain text; **WordPad** = basic rich text. Disk Cleanup frees space; **never defragment an SSD** (use TRIM); **xcopy /s** copies subdirectories.

Example 60.14.1 (Tracing a mixed item). *An item lists statements and asks which are true: (I) Preemption is one of the four necessary conditions for deadlock. (II) Priority inheritance reduces priority inversion. (III) Aging helps prevent starvation. Statement I is **false**: preemption is a recovery method; the condition is no preemption. Statements II and III are **true**. The answer is "II and III only".*

Exercises

1. List the four necessary conditions for deadlock, and name one method of recovering from a deadlock.
2. Explain priority inversion and state which technique resolves it.
3. Distinguish starvation from deadlock, and name the standard remedy for starvation.
4. What is an RTOS? Give the difference between a hard and a soft real-time system.
5. State the meaning of **/root** and **/home** in Linux, and name the default file system of Windows and of Linux.
6. Explain why an SSD should not be defragmented, and state what **xcopy /s** does.

Solutions

1. Mutual exclusion, hold and wait, no preemption, and circular wait. A deadlock can be recovered from by resource preemption (taking a resource away from one process and giving it to another).
2. Priority inversion is when a high-priority task waits because a low-priority task holds a needed resource, while a medium-priority task runs. **Priority inheritance** resolves it: the low-priority holder temporarily inherits the high priority and releases the resource quickly.
3. A starved process is ready but repeatedly passed over, so it may wait indefinitely; a deadlocked process can never proceed because of a circular resource dependency. **Aging** — gradually raising the waiting process's priority — prevents starvation.

4. An RTOS (Real-Time Operating System) guarantees a response within a bounded time so deadlines are met. In a **hard** real-time system a missed deadline is a failure; in a **soft** real-time system a missed deadline only degrades quality.
5. `/root` is the home directory of the superuser (root); `/home` holds the home directories of ordinary users. Windows uses **NTFS**; Linux uses **ext4**.
6. An SSD has no moving parts, so defragmenting gives no speed benefit and only adds wear; SSDs use **TRIM** instead. `xcopy /s` copies a directory including its subdirectories, excluding empty ones.